



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 9

Team No.: 14

Project Title: Executable Protection System

Team Members

Roll Number	Student Name	Signature
2520030406	B.L.Praveen	
2520030163	T.Dharani Dhar	
2520030268	Rishi Allapati	

1. Abstract

Executable Protection System is a Linux-based Operating Systems and Systems Programming project that demonstrates how Linux protects executable files from being modified while they are being executed. In a multitasking environment, an executable program may be accessed by multiple processes, and allowing an active executable to be overwritten can result in corrupted or inconsistent program execution. This project addresses the problem by demonstrating the operating system's response to modification attempts on an executable that is currently in use.

The project focuses on key Operating Systems and system programming concepts such as **process execution, file handling, system calls, file descriptors, kernel error handling, and executable file protection**. System calls such as `open()` and `write()` are used to access and attempt to modify the executable file, while return values and `errno` are used to identify and report errors generated by the kernel. The `strace` utility is also used to observe the relevant system calls and understand how the operating system handles the modification attempt.

The proposed solution consists of a Linux-based application written in **C**, where a sample executable is created and executed as a running process. While the executable is active, another process attempts to open and modify the same executable, and the system response is observed and recorded. The expected outcome is a functional demonstration of Linux executable protection, providing practical understanding of **file system operations, system calls, kernel-level error handling, `errno`, and system-call tracing**, while explaining how the operating system maintains the integrity of executable programs during execution.

2. Problem Statement

In a Linux operating system, executable programs are loaded and used while they are running. If another process attempts to modify or overwrite an executable during its execution, it may result in corrupted or inconsistent program behavior. Therefore, it is important to understand how the operating system handles modification attempts on an active executable file.

This project addresses the problem by demonstrating how **Linux protects executable files from modification while they are in use**. It shows the behavior of file modification operations when an executable is actively running and observes the response provided by the operating system.

The proposed system uses Linux file-related system calls such as **open()** and **write()** to attempt access and modification of the executable. The return values of these system calls are checked to determine whether the operations succeed or fail. When an operation fails, **errno** is used to identify the reason for the failure. The **strace** utility is also used to trace and observe the relevant system calls and the responses returned by the kernel.

The project is relevant to Operating Systems and Systems Programming because it provides practical understanding of **executable file handling, file system operations, system calls, kernel error handling, file protection, and system-call tracing**. It demonstrates how Linux helps maintain the integrity of an executable while it is actively being executed.

3. Objectives

1. To understand how Linux handles executable files while they are being executed.
2. To demonstrate the protection of an active executable from modification.
3. To create and execute a sample executable program in a Linux environment.
4. To attempt to open and modify the executable while it is running.
5. To study the behavior of Linux **open()** and **write()** system calls during the modification attempt.
6. To check and report errors returned by the kernel using return values and **errno**.
7. To use **strace** to observe and analyze the relevant system calls.
8. To understand how the operating system maintains the integrity and consistency of executable files during execution.

4. Proposed Methodology

The project will be designed and implemented as a **Linux-based C application** that demonstrates the protection of an executable file while it is actively running.

1. **Executable Creation:** A sample C program will be written, compiled, and converted into an executable file using the GCC compiler.
2. **Executable Execution:** The sample executable will be started and kept running for a sufficient period to allow another process to attempt modification.
3. **File Access:** A separate C program or process will attempt to open the running executable using the **open()** system call with write-related permissions.
4. **Modification Attempt:** If the file is successfully opened, the program will attempt to modify its contents using the **write()** system call.

5. **Error Checking:** The return values of `open ( )` and `write ( )` will be checked. If an operation fails, the program will use **`errno`** and appropriate error-reporting functions to identify the reason for failure.
6. **System Call Tracing:** The `strace` utility will be used to observe the relevant system calls, their arguments, return values, and errors generated by the kernel.
7. **Operating System Response:** The response of Linux to the modification attempt will be recorded and analyzed, demonstrating how an active executable is protected from modification.
8. **Testing:** The program will be executed in a Linux terminal and tested by performing modification attempts while the executable is running and after it has stopped.
9. The overall execution flow will be:

Create Executable → Execute Program → Attempt File Modification → Check System Call Result → Identify Error using `errno` → Trace using `strace` → Observe OS Response → Analyze Protection

This methodology provides a practical demonstration of **executable file protection, file system operations, system calls, kernel error handling, and system-call tracing.**

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Execution – <code>exec ()</code> / executable loading	Used to execute the sample program and demonstrate an active executable file.
File Handling – <code>open ()</code>	Attempts to open the active executable with write-related access.
File Modification – <code>write ()</code>	Attempts to modify the contents of the executable file.
File Descriptors	Used to identify and operate on the opened executable file.
Error Handling – <code>errno</code>	Identifies the reason when a system call fails.
System Call Tracing – <code>strace</code>	Observes <code>open ()</code> , <code>write ()</code> , and other relevant system calls and their return values.
Kernel Protection – <code>ETXTBSY</code>	Demonstrates the error associated with attempting to modify an executable that is currently in use.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030268	A.Rishi	Developed the C programs and implemented executable creation and execution.
2520030163	T.Dharani Dhar	Implemented file modification attempts using <code>open()</code> and <code>write()</code> and handled errors using <code>errno</code> .
2520030406	B.L.Praveen	Performed testing using <code>strace</code> , analyzed system-call behavior, and documented the results.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating environment for developing, compiling, and executing the process communication program.
C / C++	Used to implement process creation, signal handling, communication, and process control.
GCC / G++ Compiler	Compiles the C/C++ source code into an executable program.
POSIX Libraries	Provides APIs and functions for signals, processes, and synchronization.
Linux Terminal / Shell	Used to compile, execute, test, and monitor processes and signals.

8. Expected Outcome

The project is expected to produce a functional **Linux-based executable protection demonstration** that shows how the operating system handles attempts to modify an executable while it is being executed.

1. The sample C program will be successfully compiled into an executable file.
2. The executable will be successfully started and maintained in a running state during testing.
3. Another process will attempt to open and modify the active executable using Linux system calls.
4. The program will check and display the return values of the file operation system calls.
5. When the modification is rejected, **errno** will be used to identify and report the reason for the failure.
6. The **strace** utility will clearly show the relevant system calls and the response returned by the kernel.
7. The demonstration will show the difference between attempting to modify an executable while it is running and accessing it after execution has stopped.
8. Overall, the project will provide practical understanding of **Linux executable protection, file handling, system calls, kernel error handling, errno, and system-call tracing**, while demonstrating how the operating system helps maintain executable integrity during execution.

9. GitHub Repository

GitHub Repository Link:

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name:

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature: